

Capstone Project: FinOps Forecasting Agent

Author: Claus Albuquerque

Course: Agentic AI Program: Building Autonomous Systems for Real-World Applications

Checkpoint: 1.2 - August 2, 2026.

Agent Type: Forecaster

Reasoning Loop: ReAct Across Two Specialist Agents

The system is composed of two collaborating agents — a **FinOps Agent** and an **SRE Agent** — each implementing a ReAct (Reason + Act) loop. Both follow the **Thought → Action → Observation → Thought → ...** pattern until a grounded conclusion is reached or human input is required. Neither agent operates in isolation: the FinOps Agent depends on the SRE Agent to validate whether a cost optimization is operationally safe, and the SRE Agent depends on the FinOps Agent to quantify which infrastructure concerns have meaningful financial impact.

The **FinOps Agent** reasons over FOCUS-normalized cost data (FinOps Open Cost and Usage Specification). A concrete trace: the agent detects a 34% week-over-week increase in `Compute` costs, queries `consumption_records` grouped by `resource_id` to isolate three VMs driving the spike, then delegates to the SRE Agent to assess utilization. The SRE Agent returns that CPU averages 12% and the SKU was unnecessarily upgraded 8 days ago. The FinOps Agent then forecasts savings from reverting the SKU and proposes a rightsizing recommendation routed for approval.

The **SRE Agent** reasons over infrastructure topology (resource inventory, provisioned capacity), utilization metrics (daily P95 roll-ups with `isUnderused` flags), and metric definitions. It answers: "Is this resource right-sized for its actual workload?" Before recommending action, the SRE Agent checks dependency relationships — availability sets, load balancers, database connections — to confirm changes are safe.

The agents collaborate through structured delegation. When the FinOps Agent needs infrastructure context, it delegates to the SRE Agent with specific resource identifiers and a question. The SRE Agent's structured assessment becomes an Observation in the FinOps Agent's outer loop. The reverse also occurs: the SRE Agent delegates cost quantification to the FinOps Agent, ensuring infrastructure findings translate into dollar-denominated recommendations.

Memory Architecture: Short-Term and Long-Term

The system requires both memory types.

Short-term memory is the scratchpad of a single ReAct episode: the accumulated reasoning trace (each Thought must reference prior Observations), conversational context (resolving follow-up questions like "What about the same team last month?"), and intermediate results. It is implemented as the LLM's context window, with summarization for investigations that risk exceeding limits.

Long-term memory is implemented in PostgreSQL, serving as both a data tool and persistent knowledge store. It holds four categories: (1) **optimization history** — every recommendation's status and outcome, preventing re-proposal of rejected suggestions; (2) **anomaly context** — root-cause resolutions for past cost spikes, enabling pattern matching; (3) **infrastructure baselines** — interpreted usage patterns (e.g., "this VM runs batch jobs at night, so low daytime CPU is expected"), preventing false-positive underuse alerts; and (4) **user preferences** — per-team alert thresholds and engagement patterns.

Long-term memory is consulted at three critical points: before generating recommendations (to avoid repeating rejected ones), during anomaly investigation (to match known patterns), and when interpreting utilization data (to contextualize raw metrics against established baselines).

External Tool Usage: Grounding Reasoning in Real Data

Tools overcome three LLM limitations: **data retrieval**, **numerical computation**, and **real-time state access**.

The agents invoke the **Cloud Cost Management API** for billing data, **Cloud Monitor Metrics API** for utilization time-series, and **Cloud Resource Graph** for resource topology and dependencies. The PostgreSQL database serves double duty — as a retrieval tool (querying historical cost records and utilization summaries) and as a memory persistence tool (storing optimization history and anomaly resolutions).

A concrete example: given the task *"What is the projected end-of-month spend for the data-platform team?"*, the agent (1) queries actual month-to-date spend from `consumption_records`, (2) invokes a Prophet-based forecasting model for time-series projection, (3) retrieves flagged underused resources, and (4) checks previously rejected recommendations. Each step addresses a specific limitation: retrieval, computation, state access, and persistence. Without these tools, any cost figure would be a hallucination.

How This Design Improves Over a Prompt-Only Approach

The specific failure mode this design resolves is the **groundless recommendation loop** — a prompt-only system generates suggestions that are factually wrong (hallucinated figures), operationally unsafe (ignoring dependencies), or counterproductive (re-proposing rejected ideas). Asked "How can I reduce compute costs?", a prompt-only system offers generic advice: "Consider rightsizing underutilized VMs." This design instead produces: "VM `analytics-worker-02` has averaged 12% CPU over 30 days. Downgrading from `Standard_E16s_v3` to `Standard_D4s_v3` saves ~\$847/month. SRE confirms no load-balancer dependencies. [Approve] [Dismiss]."

Each failure component is resolved: hallucinated figures are grounded through tool calls; unsafe recommendations are validated by the SRE Agent; repeated rejected suggestions are filtered by long-term memory; and generic advice is replaced by specific, scoped actions with dollar estimates and operational context.